

LangGraph Persistence

Literal Execution of Checkpointers and Stores

LangGraph provides two complementary persistence systems to retain information beyond a single graph run:

- **Checkpointers:** Persist a thread's graph state as checkpoints. Used for short-term, thread-scoped memory (e.g., conversation continuity, fault tolerance).
- **Stores:** Persist application-defined data outside the graph state. Used for long-term, cross-thread memory (e.g., user preferences, shared facts).

1. Core Concepts

Concept	Description
Checkpoint	Snapshot of the graph state at a given point in time.
Thread ID	Identifies separate checkpoint sequences, mapping to a specific execution thread.
Serde	Serialization protocol used to encode/decode checkpoint data (defaults to <code>JsonPlusSerializer</code>).

2. Initializing Checkpointers and Stores

You can compile a graph passing both a checkpointer and a store to enable simultaneous short-term and long-term memory.

```

from langgraph.checkpoint.memory import InMemorySaver
from langgraph.store.memory import InMemoryStore
from langgraph.graph import StateGraph

# Literal execution: Initialize memory savers
checkpointer = InMemorySaver()
store = InMemoryStore()

# Assume 'builder' is a StateGraph populated with nodes and edges
# builder = StateGraph(AgentState)
# ... add nodes and edges ...

# Compile graph with both persistence mechanisms
graph = builder.compile(checkpointer=checkpointer, store=store)

```

3. Production Checkpoint Savers (PostgreSQL)

While `InMemorySaver` is used for debugging, `PostgresSaver` supports production workloads with full checkpoint history.

```

from langgraph.checkpoint.postgres import PostgresSaver

# Literal execution: Synchronous PostgreSQL Saver
# conn_string = "postgresql://user:pass@localhost/db"
# with PostgresSaver.from_conn_string(conn_string) as checkpointer:
#     checkpointer.setup()
#     graph = builder.compile(checkpointer=checkpointer)

```

4. Graph Invocation with Thread Configuration

To use persistence during execution, the `thread_id` must be provided in the configurable block of the invocation parameters.

```

# Literal execution: Execute with a thread ID for state tracking
inputs = {"messages": [{"role": "user", "content": "Update the database schema."}]}

# The thread_id maps this run to a distinct checkpoint sequence
config = {"configurable": {"thread_id": "thread-456"}}

result = graph.invoke(inputs, config)

```